

Corso di Laurea in Informatica A.A. 2024-2025

Laboratorio di Sistemi Operativi

Alessandra Rossi

Esempio - struttura

utilizziamo un server web semplice (come un'applicazione Python Flask) e un client web basato su HTML/JavaScript (che può essere servito dal server Flask)

```
project-directory/  
├─ Dockerfile  
├─ docker-compose.yml  
├─ start.sh  
└─ app/  
    ├─ server.py  
    └─ static/  
        └─ index.html
```

Esempio - app/server.py

```
# app/server.py
from flask import Flask, send_from_directory

app = Flask(__name__)

# Endpoint per servire la pagina principale del client
@app.route('/')
def serve_client():
    return send_from_directory('static', 'index.html')

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Esempio - app/static/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Simple Client</title>
</head>
<body>
  <h1>Hello from the Client!</h1>
  <p>This is a simple client served by Flask.</p>
</body>
</html>
```

Esempio - start.sh

```
#!/bin/bash
```

```
# Avvia il server Flask in background  
python3 /app/server.py &  
wait
```

Esempio - Dockerfile

```
# Usa un'immagine Python come base  
FROM python:3.9-slim
```

```
# Crea una cartella per l'applicazione  
WORKDIR /app
```

```
# Copia i file dell'app nel container  
COPY app /app
```

```
# Installa Flask  
RUN pip install Flask
```

```
# Copia lo script di avvio  
COPY start.sh /start.sh  
RUN chmod +x /start.sh
```

```
# Espone la porta 5000 per il server Flask  
EXPOSE 5000
```

```
# Comando di default per eseguire lo script di avvio  
CMD ["/start.sh"]
```

Esempio - docker-compose.yml

```
version: '1.0'
```

```
services:
```

```
  web_app:
```

```
    build: .
```

```
    container_name: web_app
```

```
    ports:
```

```
      - "5000:5000" # Mappa la porta Flask all'host
```

Esempio - esecuzione

```
docker-compose up -d --build
```

<http://localhost:5000>

Esercizio 1

Usando docker-compose, PostgreSQL and pgAdmin

Esercizio 1 - Postgresql

services:

postgres:

image: postgres:latest

container_name: my_postgres

environment:

POSTGRES_USER: myuser

POSTGRES_PASSWORD: mypassword

POSTGRES_DB: mydatabase

ports:

- "5432:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

volumes:

postgres_data:

`docker compose up`

Esercizio 1

services:

pgadmin:

image: dpage/pgadmin4:latest

container_name: my_pgadmin

environment:

PGADMIN_DEFAULT_EMAIL: admin@example.com # email di login per pgAdmin

PGADMIN_DEFAULT_PASSWORD: admin # password di login per pgAdmin

ports:

- "8080:80" # collega la porta 8080 del host alla porta 80 del contenitore

volumes:

- pgadmin_data: /var/lib/pgadmin

volumes:

pgadmin_data:

`docker compose up`

<http://localhost:8080>

admin@example.com e admin

Soluzione 1 - Progetto

La struttura del progetto

```
project-directory/  
├─ Dockerfile  
├─ docker-compose.yml  
├─ .env  
└─ start.sh
```

Soluzione 1 - env

Possono andare in .env file oppure in docker-compose.yml

#PostgreSQL

POSTGRES_USER=admin

POSTGRES_PASSWORD=admin_password

POSTGRES_DB=mydatabase

pgAdmin

PGADMIN_DEFAULT_EMAIL=admin@admin.com

PGADMIN_DEFAULT_PASSWORD=admin

Soluzione 1 - Dockerfile

```
# Usa un'immagine base Debian
FROM debian:latest

# Variabili d'ambiente per PostgreSQL e pgAdmin
ENV POSTGRES_USER=${POSTGRES_USER}
ENV POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
ENV POSTGRES_DB=${POSTGRES_DB}
ENV PGADMIN_DEFAULT_EMAIL=${PGADMIN_DEFAULT_EMAIL}
ENV PGADMIN_DEFAULT_PASSWORD=${PGADMIN_DEFAULT_PASSWORD}

# Aggiorna pacchetti e installa PostgreSQL, pgAdmin e dipendenze
RUN apt-get update && \
    apt-get install -y postgresql postgresql-contrib wget curl \
        python3 python3-pip && \
    pip3 install pgadmin4

# Copia lo script di avvio nel container
COPY start.sh /start.sh
RUN chmod +x /start.sh

# Espone le porte per pgAdmin e PostgreSQL
EXPOSE 5432 80

# Comando di default per eseguire lo script di avvio
CMD ["/start.sh"]
```

Soluzione 1 - docker-compose.yml

services:

db_pgadmin:

build: .

container_name: db_pgadmin

environment:

POSTGRES_USER: \${POSTGRES_USER}

POSTGRES_PASSWORD: \${POSTGRES_PASSWORD}

POSTGRES_DB: \${POSTGRES_DB}

PGADMIN_DEFAULT_EMAIL: \${PGADMIN_DEFAULT_EMAIL}

PGADMIN_DEFAULT_PASSWORD: \${PGADMIN_DEFAULT_PASSWORD}

ports:

- "5432:5432" # Porta PostgreSQL

- "8080:80" # Porta pgAdmin

volumes:

- postgres_data:/var/lib/postgresql/data

volumes:

postgres_data:

Soluzione 1 - avvio

```
#!/bin/bash
```

```
# Avvia PostgreSQL in background  
service postgresql start
```

```
# Configura PostgreSQL  
sudo -u postgres psql -c "CREATE USER ${POSTGRES_USER} WITH PASSWORD '${POSTGRES_PASSWORD}';" || true  
sudo -u postgres psql -c "CREATE DATABASE ${POSTGRES_DB} OWNER ${POSTGRES_USER};" || true
```

```
# Configura pgAdmin (server mode disabilitato per singolo utente)  
mkdir -p /root/.pgadmin  
echo "SERVER_MODE = False" >> /root/.pgadmin/config_local.py  
echo "DEFAULT_SERVER = '0.0.0.0'" >> /root/.pgadmin/config_local.py  
echo "DEFAULT_SERVER_PORT = 80" >> /root/.pgadmin/config_local.py
```

```
# Avvia pgAdmin in background  
python3 -m pgadmin4 &
```

```
# Mantiene il container in esecuzione  
wait
```

Soluzione 1 - esecuzione

`docker compose up —build`

pgAdmin sarà disponibile su `http://localhost:8080`.

PostgreSQL sarà accessibile sulla porta 5432 con le credenziali definite nel file `.env`.



Università degli Studi di Napoli Federico II

THANK YOU FOR YOUR ATTENTION

TRUST ME ...
I AM A ROBOT!

